

WizardCD — Project Overview & Architecture

Version: 1.0 | Updated: 2026-03-11

Table of Contents

- [1. What is WizardCD?](#)
- [2. Name Suggestions](#)
- [3. System Architecture](#)
- [4. End-to-End Deployment Flow](#)
- [5. Deployment Package on Target VM](#)
- [6. Component Reference](#)
- [7. Environment Configuration](#)
- [8. Key File Paths](#)

1. What is WizardCD?

WizardCD is a **self-hosted Java application deployment framework** built for teams that deploy Spring Boot (and other Java) applications directly to Linux VMs — without Docker, Kubernetes, or CI/CD pipelines like Jenkins.

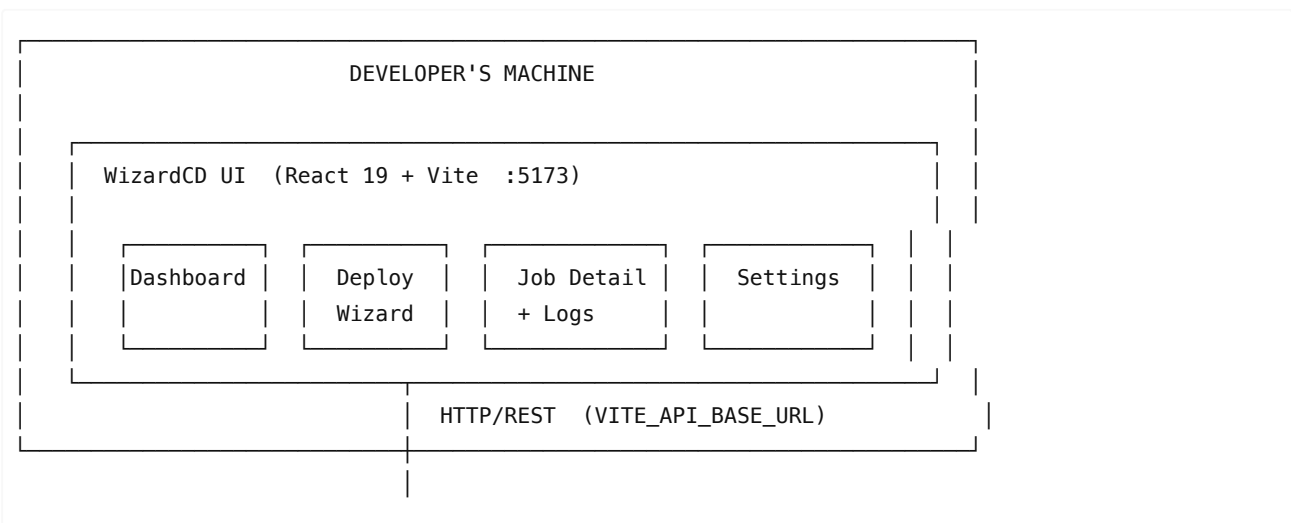
It gives teams a clean web UI to configure, trigger, track, and audit deployments across multiple environments (SIT, UAT, PROD) in a controlled, repeatable way.

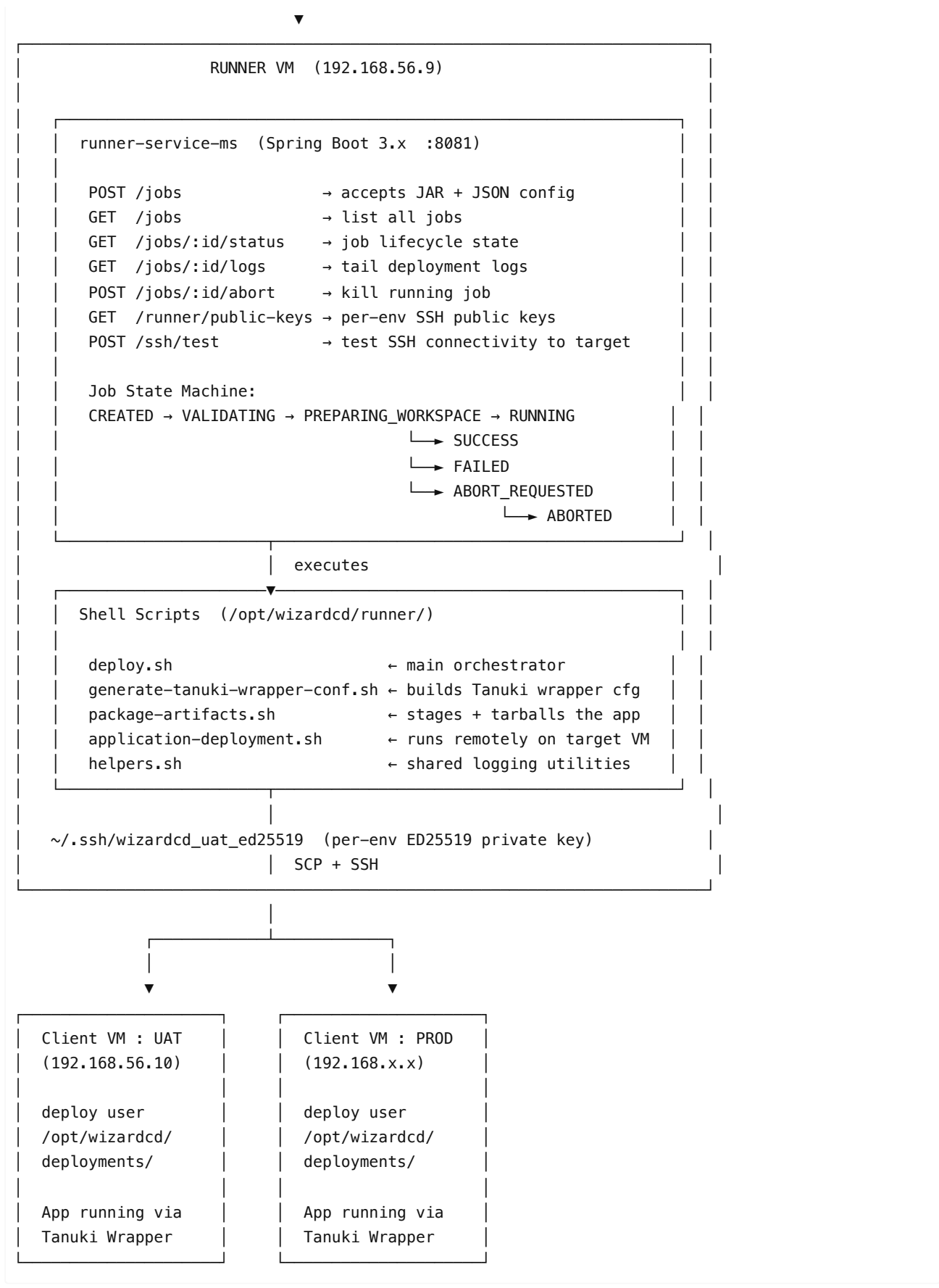
i **WizardCD is not a SaaS.** It is installed on your own infrastructure. The runner service sits inside your network and has SSH access to your target servers. Think of it as a lightweight, self-contained alternative to tools like Rundeck or Ansible Tower — purpose-built for Java VM deployments.

Key Characteristics

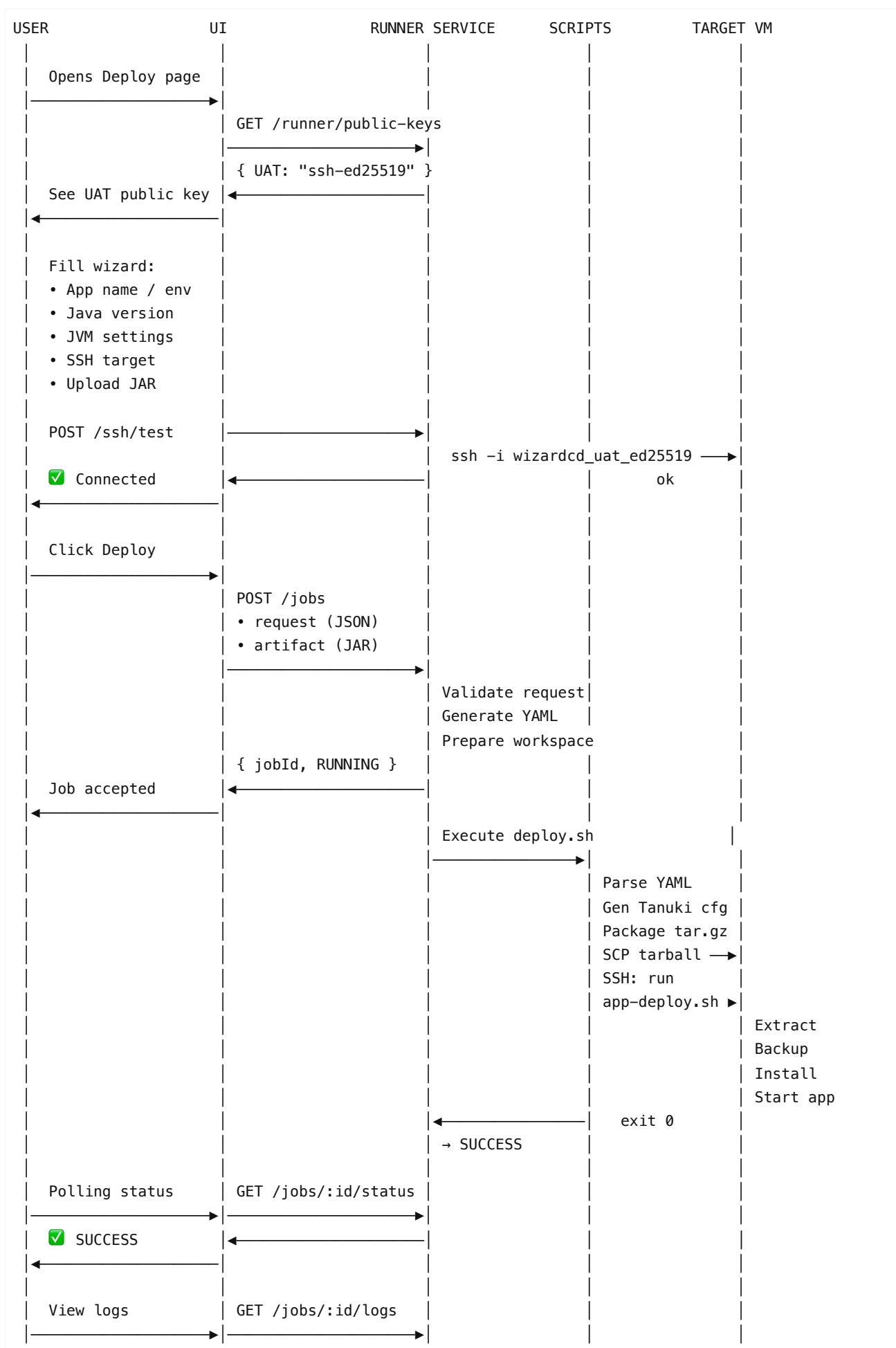
- Self-hosted and network-contained
- Multi-environment aware — SIT / UAT / PROD
- Java and Spring Boot focused
- VM-based deployments via SSH — no container orchestration required
- Tanuki Service Wrapper for process lifecycle management on target VMs
- Per-environment ED25519 SSH key isolation
- Real-time job tracking and log streaming
- Backup-before-deploy with configurable retention

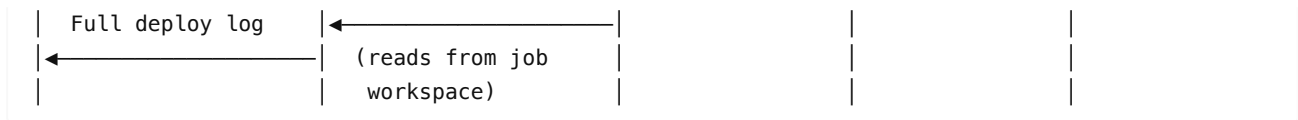
2. System Architecture





3. End-to-End Deployment Flow





4. Deployment Package on Target VM

When a deployment completes, the following structure is created on the target VM:

```
/opt/wizardcd/deployments/  
└─ myapp/  
    ├── bin/  
    │   ├── myapp.jar           ← the application JAR  
    │   ├── myapp-wrapper.sh   ← Tanuki start/stop/restart script  
    │   └── wrapper            ← Tanuki native binary  
    ├── conf/  
    │   └── myapp-uat.conf      ← JVM + logging config (auto-generated)  
    ├── lib/  
    │   ├── wrapper.jar  
    │   └── libwrapper.so  
    └── backup/  
        └── myapp-uat-20260311.tar.gz ← previous version archive
```

The application runs as a **Tanuki Service Wrapper** process, which provides:

- Automatic restart on crash
- JVM memory and tuning control
- Log rotation with configurable size and retention
- Clean start / stop / restart lifecycle
- No systemd unit required per deployed application

5. Component Reference

UI Pages

Page	Purpose
Dashboard	Job metrics and trends — total, running, successful, failed, aborted
Deploy Wizard	4-step form: App Identity → Java/JVM → Runtime → SSH Target
Job Detail	Live status, execution state, full log viewer, abort button
Settings	Application-level configuration

Runner API Endpoints

Method	Path	Description
POST	/jobs	Submit a deployment — multipart (JSON config + JAR artifact)
GET	/jobs	List all jobs — optional ?status= filter
GET	/jobs/:id/status	Fetch current lifecycle and execution state
GET	/jobs/:id/logs	Tail last N log lines from deploy.log
POST	/jobs/:id/abort	Request abort of a running job

GET	/jobs/summary	Dashboard metrics — totals and trends
GET	/runner/public-keys	Per-environment SSH public keys (SIT / UAT / PROD)
POST	/ssh/test	Test SSH connectivity from runner to a target VM

Job Lifecycle States

State	Description
CREATED	Job accepted by the runner, not yet validated
VALIDATING	Input YAML and parameter validation in progress
PREPARING_WORKSPACE	Job workspace directories and input files being staged
RUNNING	deploy.sh is currently executing
SUCCESS	Deployment completed successfully (exit code 0)
FAILED	Deployment failed — non-zero exit code or exception
ABORT_REQUESTED	Abort requested while job is in RUNNING state
ABORTED	Process terminated following abort request

SSH Key Model

One ED25519 key pair per environment, auto-generated by the runner on first use.

Environment	Private Key Location (Runner VM)	Purpose
SIT	~/.ssh/wizardcd_sit_ed25519	SSH into SIT target servers
UAT	~/.ssh/wizardcd_uat_ed25519	SSH into UAT target servers
PROD	~/.ssh/wizardcd_prod_ed25519	SSH into PROD target servers

⚠️ The private key **never leaves the runner VM**. Only the public key is authorised on target servers. The UI only ever displays the public key.

6. Environment Configuration

Environment	UI Colour	Runner VM	Client VM
SIT	 Blue	192.168.56.9:8081	Local / dev machine
UAT	 Yellow	192.168.56.9:8081	192.168.56.10
PROD	 Purple	192.168.56.9:8081	192.168.x.x (TBD)

Profile Resolution Order

Priority highest → lowest:

1. JVM argument — `--spring.profiles.active=X`
2. Environment variable — `SPRING_PROFILES_ACTIVE` (set automatically by `start-runner.sh`)
3. Bundled default — `spring.profiles.active` inside `application.yaml` within the JAR

7. Key File Paths

Runner VM — `/opt/wizardcd/`

Path	Description
<code>runner-service-ms-0.0.1-SNAPSHOT.jar</code>	Application JAR
<code>runner/deploy.sh</code>	Main deployment orchestrator
<code>runner/application-deployment.sh</code>	Remote execution script (runs on target VM via SSH)
<code>runner/generate-tanuki-wrapper-conf.sh</code>	Generates Tanuki wrapper <code>.conf</code> and <code>.sh</code> files
<code>runner/start-runner.sh</code>	Service startup — auto-resolves active profile from JAR
<code>workspace/jobs/<jobId>/input/</code>	Job input — JAR + generated YAML config
<code>workspace/jobs/<jobId>/build/</code>	Packaged deployment tarball
<code>workspace/jobs/<jobId>/logs/</code>	<code>deploy.log</code> , <code>package.log</code> , <code>ssh.log</code>
<code>logs/runner-service-ms.log</code>	Runner service log (rolling, 30-day retention)
<code>/home/deploy/.ssh/wizardcd_*_ed25519</code>	Per-environment SSH key pairs
<code>/etc/systemd/system/wizardcd-runner.service</code>	systemd service unit

Target VM — `/opt/wizardcd/deployments/<appname>/`

Path	Description
<code>bin/<appname>.jar</code>	The deployed application JAR
<code>bin/<appname>-wrapper.sh</code>	Tanuki start / stop / restart script
<code>bin/wrapper</code>	Tanuki native binary
<code>conf/<appname>-<env>.conf</code>	Auto-generated Tanuki wrapper configuration
<code>lib/wrapper.jar</code>	Tanuki Java library
<code>lib/libwrapper.so</code>	Tanuki native shared library
<code>backup/</code>	Previous deployment archives